Excellent

final : 29 out of 32

Imperial College London

Imperial College London

# Assignment 3

Introduction to Machine Learning

P1: 4/4

P3 (a,d): 11.5/12

Nicolas Perez
3-6-2017
CID: 00931920

# *Non-plagiarism Pledge*

*I, Nicolas Perez, pledge that this assignment is completely my own work, and that I did not take, borrow or steal work from any other person, and that I did not allow any other person to use, have, borrow or steal portions of my work. I understand that if I violate this honesty pledge, I am subject to disciplinary action pursuant to the appropriate sections of Imperial College London.*

*Aspects of this coursework were discussed together with Aaron Low*

# Contents

Non-plagiarism Pledge ..... 1
Problem 1 ..... 3
Problem 2 ..... 4
Problem 3 ..... 6
(a) Constant base predictors ..... 6
(b) Linear Regression baseline ..... 6
(c) Linear regression with transformed features ..... 7
(d) Collaborative filtering ..... 8
Appendix ..... 10
References ..... 11

## Problem 1

**Problem 1** (Approximate ERM). Suppose you need to solve a binary classification problem with the binary error (i.e., the error is zero if the prediction is correct and 1 otherwise). Assume you are given data $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ drawn i.i.d. from the true data distribution. For any $\varepsilon \geq 0$, give a bound on the test error of any hypothesis $h_\varepsilon$ that is $\varepsilon$-optimal on the training set $\mathcal{D}$, that is, it satisfies $\widehat{R}_n(h_\varepsilon) \leq \varepsilon + \min_{h \in \mathcal{H}} \widehat{R}_n(h)$;[1] more precisely, give a high-probability upper bound on $R(h_\varepsilon) - \min_{h \in \mathcal{H}} R(h)$.

Let $h$ be a hypothesis in the hypothesis class $\mathcal{H}$. Then let's define $g$ and $h^*$ as:

$$g \in argmin(\hat{R}_n(h)) \qquad \textit{(def. 1)}$$

$$h^* \in argmin(R(h)) \qquad \textit{(def.2)}$$

That is, $g$ is the Empirical Risk Minimizer Hypothesis on the n-point training set $\mathcal{D} \subseteq \mathcal{X}$ and $h^*$ is the hypothesis with which minimises the test error on the set $\mathcal{X}$.

Then for any $\varepsilon \geq 0$ we have a set of hypothesis $\mathcal{H}_\varepsilon \subseteq \mathcal{H}$ which are $\varepsilon$-optimal on $\mathcal{D}$. This means that any $h_\varepsilon$ in this set satisfy

$$\hat{R}_n(h_\varepsilon) \leq \varepsilon + min_{h \in \mathcal{H}} \hat{R}_n(h) \qquad \textit{(eq. 1)}$$

According to the *def. 1* we have that *eq.1* is equivalent to $\hat{R}_n(h_\varepsilon) \leq \varepsilon + \hat{R}_n(g)$. This means that $h_\varepsilon$ is any hypothesis whose training error is at most $\varepsilon$ above the training error of the ERM hypothesis.

We want to bound the following expression

$$R(h_\varepsilon) - min_{h \in \mathcal{H}} R(h)$$

This can also we written as $R(h_\varepsilon) - min_{h \in \mathcal{H}} R(h) = R(h_\varepsilon) - R(h^*)$ using *def.2*

Thus, we want to bound how much worse would be the test error in case we choose any of the hypothesis $h_\varepsilon$ compared to the test error obtained when choosing $h^*$.

We have that

$$R(h_\varepsilon) - R(h^*) = R(h_\varepsilon) - \hat{R}_n(h_\varepsilon) + \hat{R}_n(h_\varepsilon) - \hat{R}_n(g) + \hat{R}_n(g) - \hat{R}_n(h^*) + \hat{R}_n(h^*) - R(h^*)$$

We can bound each two terms of the sum the following way

1. First term $R(h_\varepsilon) - \hat{R}_n(h_\varepsilon)$ we use the Vapnik-Chervonenkis Inequality which bound with a probability of at least $1 - \delta_1$

$$R(h_\varepsilon) - \hat{R}_n(h_\varepsilon) \leq \sqrt{\frac{8}{n} \ln\left(\frac{4 m_{\mathcal{H}}(2n)}{\delta_1}\right)} \leq \sqrt{\frac{8 d_{VC}}{n} \ln(2n+1) + \frac{8}{n} \ln\left(\frac{4}{\delta_1}\right)}$$

2. The second term $\hat{R}_n(h_\varepsilon) - \hat{R}_n(g)$ is, as stated above, equivalent to *eq.1*. Thus:

$$\hat{R}_n(h_\varepsilon) - \hat{R}_n(g) \leq \varepsilon$$

3. The third term is $\hat{R}_n(g) - \hat{R}_n(h^*) \leq 0$ according to the definition of $g$ (the training error of $g$ n the training set $\mathcal{D}$ is less than that of any of the other hypothesis).
4. Finally, the fourth term, $\hat{R}_n(h^*) - R(h^*)$ can be bounded using Hoeffding inequality which bound with a probability of at least $1 - \delta_2$

$$\hat{R}_n(h^*) - R(h^*) \leq \sqrt{\frac{1}{2n}\ln\left(\frac{2}{\delta_2}\right)}$$

4/4

Putting everything together gives that with a probability of $1 - \delta_1 - \delta_2$:

$$R(h_\varepsilon) - R(h^*) \leq \sqrt{\frac{8d_{VC}}{n}\ln(2n+1) + \frac{8}{n}\ln\left(\frac{4}{\delta_1}\right)} + \sqrt{\frac{1}{2n}\ln\left(\frac{2}{\delta_2}\right)} + \varepsilon$$

## Problem 2

**Problem 2 (Optimal predictor for the absolute error).** Given data points $y_1, \ldots, y_n \in \mathbb{R}$, find a value $m$ minimizing the average absolute error $\frac{1}{n}\sum_{i=1}^{n}|y - y_i|$.

First, without loss of generalization, we can index the elements $y_i \in \mathbb{R}$ so that they are ordered. That is $y_1 \leq y_2 \leq \cdots \leq y_n$.

Then, we define the function $f(y) = \frac{1}{n}\sum_{i=1}^{n}|y - y_i|$.

If we evaluate the function $f$ on a $y \leq y_1$, (that is a $y$ that is smaller than all $y_i$) then as $y$ increases each term of $f(y)$ decreases at least until $y$ reaches $y_1$. That is $f(y_1) < f(y)$ for all $y < y_1$.

Then if we evaluate the function on $y + \varepsilon$, with $\varepsilon > 0$ so that $y_k \leq y \leq y + \varepsilon \leq y_{k+1}$ then

$$\begin{aligned} n \cdot f(y+\varepsilon) &= \sum_{i=1}^{k}(y + \varepsilon - y_i) + \sum_{i=k+1}^{n}(y_i - (y+\varepsilon)) \\ &= k\varepsilon + \sum_{i=1}^{k}(y - y_i) - (n-k)\varepsilon + \sum_{i=k+1}^{n}(y_i - y) \\ &= (2k-n)\varepsilon + \sum_{i=1}^{k}(y - y_i) + \sum_{i=k+1}^{n}(y_i - y) \\ &= (2k-n)\varepsilon + nf(y) \end{aligned}$$

We can the write

$$f(y+\varepsilon) - f(y) = \frac{(2k-n)\varepsilon}{n}$$

This difference is negative if $2k < n$, it is zero if $2k = n$ and positive if $2k > n$. Then in the interval $[y_k, y_{k+1}]$ the function $f(y)$, with $y_k \leq y \leq y_{k+1}$ behaves in the following way:

$$f(y)\begin{cases} decreases\ if\ 2k < n \\ constant\ if\ 2k = n \\ increases\ if\ 2k > n \end{cases}$$

This means that as $y$ moves from $-\infty$ to $+\infty$ the function will decrease and then increase and the minimum will be found in the interval $[y_k, y_{k+1}]$ where $2k = n$.

We need to consider now two possibilities.

**Even $n$.** Then $k = \frac{n}{2}$. This means that $m$, the value that minimises $f$ is in the interval $[y_{n/2}, y_{n/2+1}]$. That is $m \epsilon [y_{n/2}, y_{n/2+1}]$. All the values in this interval minimise $\frac{1}{n}\sum_{i=1}^{n}|y - y_i|$. We can take for example the average value between these two $m = \frac{y_{n/2}+y_{n/2+1}}{2}$

**Odd $n$.** Then we can write $n = 2p + 1$, where $p \epsilon \mathbb{N}$ and then $\frac{n}{2} = p + 1/2$. Then we must have that $y_k = y_{p+1/2}$ and $y_{k+1} = y_{p+3/2}$. These two values are between $y_p$ and $y_{p+2}$. Thus, we can take $m = y_{p+1}$ as our minimizer, that is, the value in the middle of the set of n-points.

In conclusion, we have found a set of values that minimise $\frac{1}{n}\sum_{i=1}^{n}|y - y_i|$ which is $[y_k, y_{k+1}]$ where $2k = n$. However, since we only need to pick one we can just pick the following

$$m = \begin{cases} \frac{y_{n/2} + y_{n/2+1}}{2}, & if\ n\ even \\ y_{p+1}, & if\ n\ is\ odd \end{cases}$$

Where $n = 2p + 1$, where $p \epsilon \mathbb{N}$ for the odd case.

This is the standard definition of the median for and even number of points (i.e. the average between the middle values) and for an odd number of points (i.e. value in the middle).

## Problem 3

### (a) Constant base predictors

Firstly, we will provide a rating prediction for any movie (independent of the user) and for any user (independent of the movie). We will use the mean as a predictor. That is, each user rating for any movie will be predicted using the mean of ratings he has given so far and each movie rating for any user will be predicted using the mean of all the users that have rated it. We will then compare how well this works. In addition to this, since there are some movies that have not been rated in the training set but that may have been rated in the testing set we will give them a rate of 2.25 which is the average rating a movie can have. Choosing this value is arbitrary, nevertheless, the results do not change in any statistically meaningful way by choosing any value in the range rating 0.5-5.

In order to compare these two different ways of predicting the ratings we will obtain the Mean Squared Error (MSE) of all the ratings in the training and in the test sets: $MSE = \frac{1}{n}\sum_{i=1}^{n}(r_i - \hat{r}_i)^2$, where $n$ is the number of points on the training or test set. We will use this MSE from now on to quantify and compare how well the different learning methods we try behave.

| Learning Strategy | MSE on Training Set | MSE on Test Set |
|---|---|---|
| **Averaging user ratings** | 0.9108 | 0.9269 |
| **Averaging movie ratings** | 0.7851 | 1.0348 |

We see that the training error is closer to the test error when using the user rating average. It also gives a smaller test error compared to averaging the movie rating. Thus, we would choose using the user ratings rather than the movie ratings.

### (b) Linear Regression baseline

**Linear Regression.** The next learning strategy will consist on using Linear Regression to estimate the ratings based on some movies features $V$. Each user will have a vector $u_i$ which multiplied by the feature $v_j$ for a movie $j$ will give a prediction of the rating for that movie and that user $\hat{r}_{ij}$. The result of this is

| Learning Strategy | MSE on Training Set | MSE on Test Set |
|---|---|---|
| **Linear Regression** | 2.6116 | 3.2073 |

**Normalised Linear Regression.** This result is worse than that obtained using the constant predictors. It will probably improve if we normalize the movie features associated to a user so that each feature is centred and has a standard deviation of 1. In addition, the ratings for that user will be centred too. Then, when testing, we shift back the resulting rating by adding the corresponding mean for that user. The results are the following:

| Learning Strategy | MSE on Training Set | MSE on Test Set |
|---|---|---|
| **Normalised Lin REG** | 0.7068 | 1.0162 |

**Ridge Regression.** There is a noticeable improvement on the results but yet it is not as good as the constant base predictors. There is still a big gap between the error on training and on testing. This suggest that we may be overfitting the data. In order to fix this, we will use Ridge Regression. This way we will penalize the modulus of the weighting vector associated to each user, $u_i$. That is, we minimize $\hat{R}_n(u_i) + \frac{\lambda}{n}\|u_i\|^2$.

In order to choose the parameter $\lambda$ for ridge regression we use cross validation. In particular, leave-one-out cross validation. That is, we will partition the training set leaving one point out that we will use to obtain the validation error. We repeat this process so that we validate with all the points in the training set. Then we obtain the cross-validation error as the average of the validation errors (which are just the MSE of the validation set). By repeating this with several $\lambda$ values we can choose which one is the best for a particular user. Thus, we will get the optimal value of $\lambda_i$ for each user $i$.

The initial set of values of $\lambda$ to be tested will be logarithmically distributed between 0.0001 and 1000 (`logspace(-4,3,10)`) since, in principle, we do not expect $\lambda$ to be very large. This gives the following results.

| Learning Strategy | MSE on Training Set | MSE on Test Set |
|---|---|---|
| **Ridge Regression** | 0.8007 | 0.9072 |

This result is better than that obtained for the constant base predictors. It is also very noticeable how the difference between the training and test errors has reduced which means that we have reduced the overfitting. However, most of the $\lambda_i$ values we obtained through cross-validation are rather large (in the order of 100) as can be seen in the figure on the right.

That means that the modulus of the weighting vectors was indeed way larger than required and the data was being overfitted. We will then distribute the values initial set of $\lambda$ to be tested to 10 to 1000 (`linspace(10,1000,25)`). This will allow to better choose $\lambda$ (the number of $\lambda s$ to choose from in the optimal interval [10,1000] is larger) After doing this we get some improvement on the results:

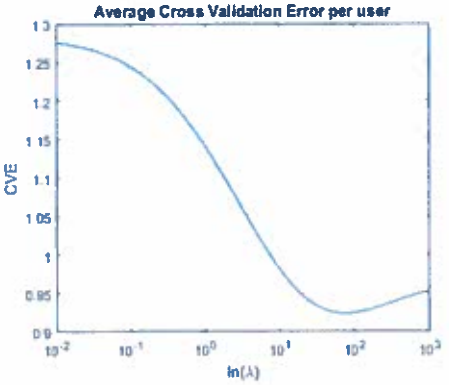


*Figure 1: Average CVE using Ridge Regression*

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Ridge Reg with optimal initial $\lambda$ set** | 0.7778 | 0.8834 |

**Lasso Penalty.** We can also try a different penalization. Instead of penalizing the square of the modulus of the weighting vectors we can penalize the absolute sum of its components. That is, a Lasso Penalization. We test several initial set of $\lambda$ as we did before and we find here that the region which gives the best results is from 0.01 to 1000 (`logspace(-2,3,100)`). However, this results in a worse outcome.

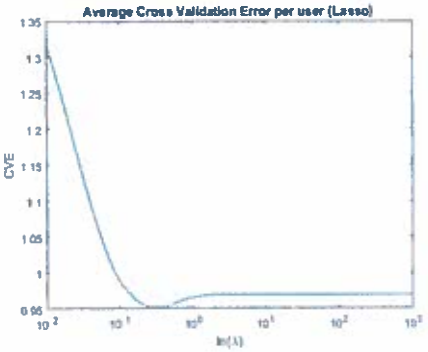


*Figure 2: Average CVE using Lasso Penalty*

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Lin Reg with LASSO penalty** | 0.7908 | 0.8941 |

## (c) Linear regression with transformed features

**PCA.** The objective now is to find a non-linear transformation that improves the results obtained so far. However, let's first try to do a Principal Component Analysis (which is a linear transformation) to try to reduce the features into a set of linearly uncorrelated variables and see if the result improves that way. We will do this with an initial set of $\lambda$ from 0.0001 to 1000 since we do not know which ones are going to work the best this time. The result obtained I the following:

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Ridge Regression after PCA features** | 0.7763 | 0.8923 |

Using PCA on the features reduces the number of features from 18 to 16. However, the outcome shows almost no improvement compared to the previous results.

**Interaction.** What we will try now is using the standard MATLAB function `x2fx()` which allows to perform an interaction non-linear transformation. That is, the output features will be all the possible products of each pair of input features ($i.e.\ x = (x_1, x_2, \dots, x_{18}) \rightarrow z = (x_1x_2, x_1x_3, \dots, x_{17}x_{18})$). There will be no pure quadratic features created since the features are binary and thus, squaring them will not have any effect. The results show no improvement and we can see in the fig. 7 in Appendix how the optimal values of $\lambda$ are even larger than those obtained before (as expected sin now the features vector has more components).

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Ridge Regression After interaction transformation** | 0.6814 | 0.9169 |

**Legendre.** The interaction transformation increases the number of features which is something that, in principle, we do not want to do since it may result in overfitting the training set. Thus, we will try to use a non-linear transformation which reduces the number of features. We will try the following, for each feature we will obtain its Legendre polynomials up to order 5. Then we will add all the new features of the same polynomial order. This means that we will end up with 6 features per movie. The results of both, test and training datasets

of this transformation are worst which suggest that we are underfitting the data. (Results of Ridge regression are on fig. 8 on the Appendix)

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Ridge Regression After Legendre transformation** | 0.8929 | 0.9115 |

**Correlation.** Finally, we will try to measure how relevant each feature is for each user. We can do this by taking the dot product of each feature with the ratings associated to that user. This will give a set of coefficients that will reflect how correlated each of them is to the rating. Then we will multiply each feature by this coefficient. This will boost those coefficients more correlated to the ratings. We should notice however that this is not a non-linear transformation, thus, after boosting the features we will perform the Legendre transformation described above. (Results of Ridge regression are on fig. 9 on the Appendix)

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Correlation and Legendre transformation** | 0.8957 | 0.9062 |

After trying all these non-linear transformations, we can conclude that none of them improve the results obtained using Ridge Regression on the features directly. In addition to this non-linear transformations more transformations using Legendre polynomials have been tested without any improvement.

## (d) Collaborative filtering

**Collaborative Filter - Gradient Descent.** The last approach that we will take will be learning simultaneously the movies features $v_j$ for all $n$ movies and the users weights $u_i$ for all $m$ users. We want to minimise the error $\hat{R}_N(u_i, v_j) = \frac{1}{2}\sum_{i,j}^{n,m}(r_{ij} - u_i^T v_j)^2 + \frac{\lambda}{2}\|v_j\|^2 + \frac{\lambda}{2}\|u_i\|^2$. If we store these vectors in the matrices $U_{mxK}$ and $V_{nxK}$ and the ratings in the matrix $r_{nxm}$ then, we can minimise $\hat{R}_N$ by updating the matrices in the following way $U_{t+1} = U_t - \eta\nabla U_t$ *where* $\nabla U_t = \sum_{i,j}^{n,m}(r - U_t V_t^T .* D)V_t + \lambda U_t$ and similarly, $V_{t+1} = V_t - \eta\nabla V_t$ *where* $\nabla V_t = \sum_{i,j}^{n,m}(r - V_t U_t^T .* D)U_t + \lambda V_t$. We have obtained the gradient from the partial derivative of the error function with respect to $U$ and $V$. The matrix $r$ will have the ratings centred per user (ignoring those movies not rated by the user). In order to not include those pairs $(i, j)$ with no rating associated we multiply element-wise with the matrix $D$ which has a 1 if a pair was rated and a 0 if a pair was not.

This method gives the following results when used with the learning rate $\eta = 0.001$, the maximum number of iterations set to 1000, $\lambda = 0$ and the dimension of the vectors, $K = 1$. These values were chosen after trying the algorithm a few times to get a reasonable $\hat{R}_{N,training}$ at the end (a value of $\eta > 0.002$ makes the error increase very fast instead of decrease).

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Collaborative Filter - Gradient Descent** | 0.7212 | 0.8453 |

We can see how the training and test error vary as we learn in the figure on the right. We notice that we did not choose the best possible maximum number of cycles since the minimum test error has not been reached yet. However, we only know this after looking at the test error which is something we cannot do beforehand. If we were to modify the algorithm because of the results we have seen on this test set we would be doing data snooping. This means that we would get a better result for this particular test set but we may do worse with others.

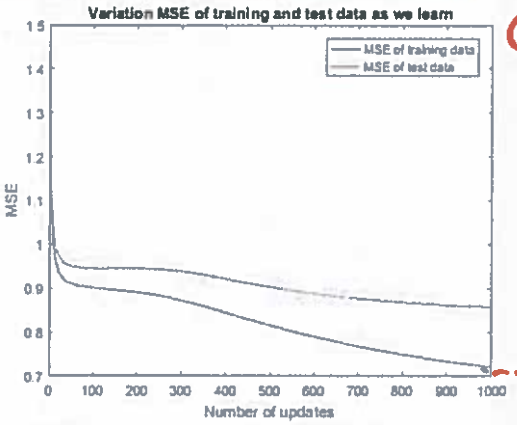


*Figure 3: Variation of MSE on the test and training data*

What we can do however is select the optimal parameters using cross validation or other similar method. However, this algorithm takes a very long time to run. Thus, we will try implement a faster version using a Stochastic Gradient Descent algorithm and use cross validation then.

monitor → training error !!!

**Collaborative Filter – Stochastic Gradient Descent.** This algorithm uses the partial derivatives at a particular, randomly chosen, point to estimate the gradients. That is, $u_{i,t+1} = u_{i,t} - \eta \nabla u_{i,t}$ *where* $\nabla u_{i,t} = (r_{ij} - u_{i,t}{}^T v_{j,t}) v_j + \lambda u_{i,t}$ and similarly for $v_{j,t+1}$. Here we iterate 350 times through all the points (i.e. $N = 70002$ points, which is the number of valid pairs $(i,j)$) and we use $\eta = 0.001$. These two values were found by trying to reduce the training error on a trial and error basis. Then, we use cross validation dividing the $N$ points into 10 partitions from which we randomly take 6000 for training and 1000 for validation. The cross validation error is obtained as in question 3b and 3c. This gives the optimal number of features is 14 ($K = 14$) and the optimal $\lambda = 0.2$ using cross validation simultaneously on both parameters. This results can be seen on the figures on the right. We used $K = [2, 3, 5, 7, 9, 10, 12, 14, 16, 18, 20]$ and $\lambda = [0.0001, 0.001, 0.01, 0.1, 0.2, 0.4, 0.6, 0.8, 1]$

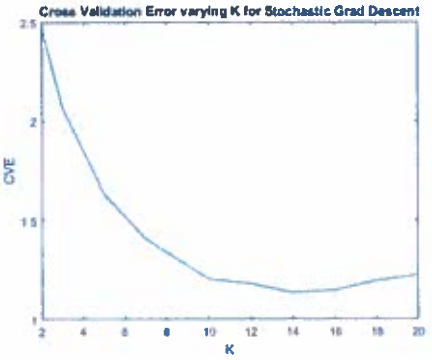



*Figure 4: Average CVE as we vary the number of features*

The best way of optimally choosing all the parameters would have been doing cross validation for all of them. However, we have not done this here due to the computational power required to evaluate all these parameters together. With the parameters considered above we get the following MSE.

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Stochastic Gradient Descent** | 0.6506 | 0.8542 |

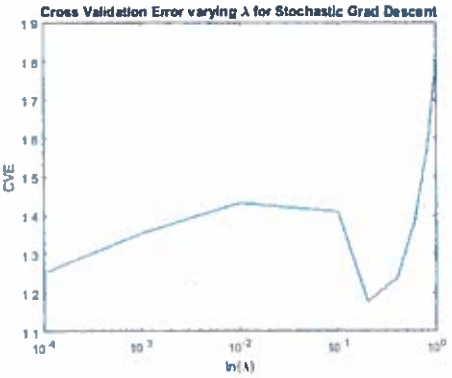
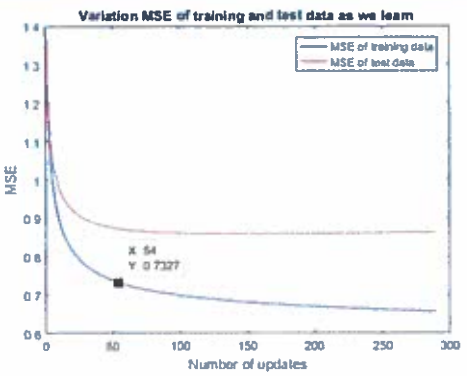



*Figure 5: Average CVE as we vary λ*

We can plot the evolution of the training and test error as well as the evolution of the average modulus of the gradient defined as $\|\nabla \hat{R}_N\| = \frac{1}{N}\left|\sum_{i,j}^{n,m}\left(\sum_{k=1}^{K} u_{i,k} + \sum_{k=1}^{K} v_{j,k}\right)\right|$, where $N$ is the total number of training points. This gives the results shown on fig. 6. From these we see two things. Firstly, we have not chosen the best maximum number of iterations since the minimum test error was reached before, this means we are slightly overfitting the data. Secondly, $\|\nabla \hat{R}_n\|$ decreases as we update the vectors. We can use $\|\nabla \hat{R}_n\|$ to stop the algorithm earlier. This would save us running time. We can see that the training error decreases slower when $\|\nabla \hat{R}_N\| < 0.006$. We could use this threshold to stop the algorithm, however, in practice, we choose a threshold smaller (0.002) to avoid stopping the algorithm too early due to the oscillation of $\|\nabla \hat{R}_N\|$. This gives the results shown in the table below.

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Stochastic Gradient Descent with Stop** | 0.6884 | 0.8577 |



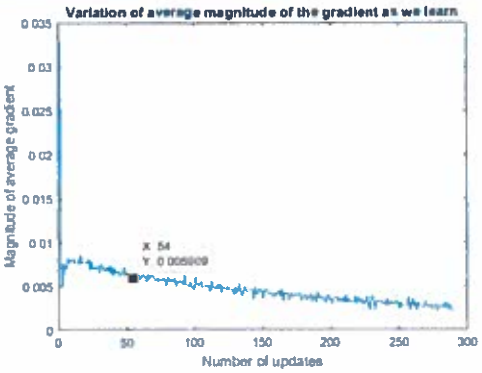




*Figure 6 :On the left, variation of MSE on the test and training data. On the right variation of the magnitude of the gradient.*

Alternatively we can approximate the derivative of the training error using the function `diff()` and stop the algorithm whenever is close enough to zero (this can be seen fig. 10 on the Appendix). We see however that even with cross validation, this algorithm does not improve the error obtained with the Non-Stochastic Gradient Descent.

8+2
10

# Appendix

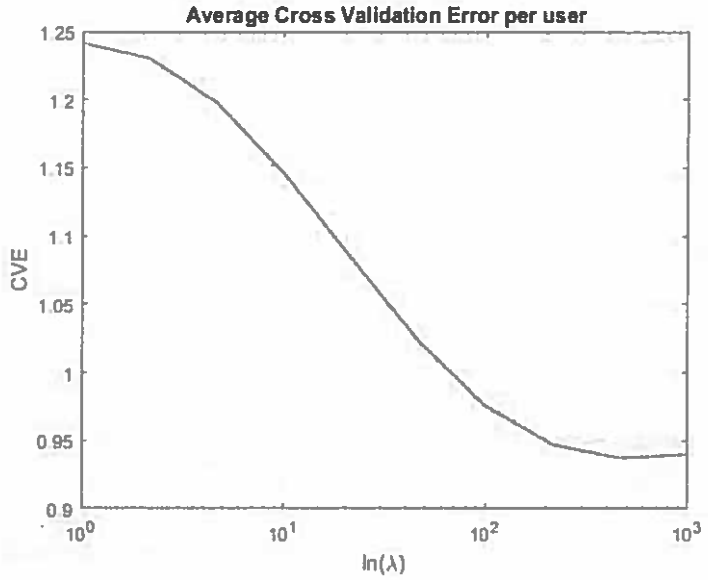



*Figure 7: Average CVE using Interaction Non-Linear Transform*

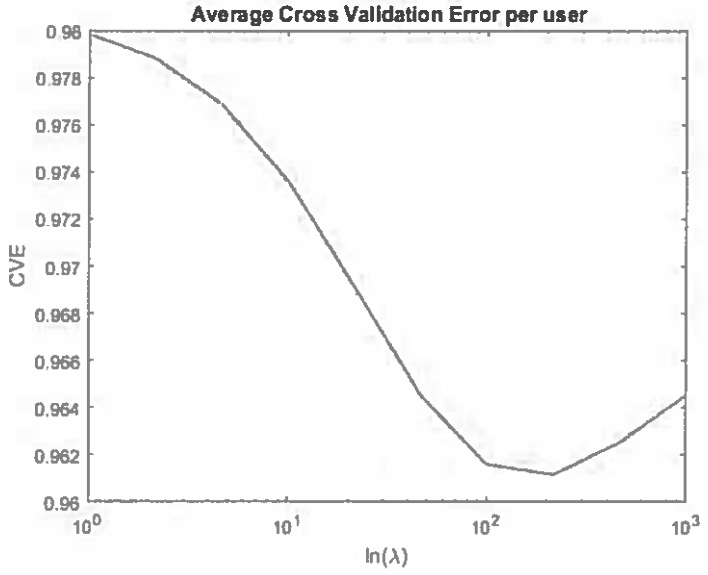



*Figure 8: Average CVE using Legendre Non-Linear Transform*

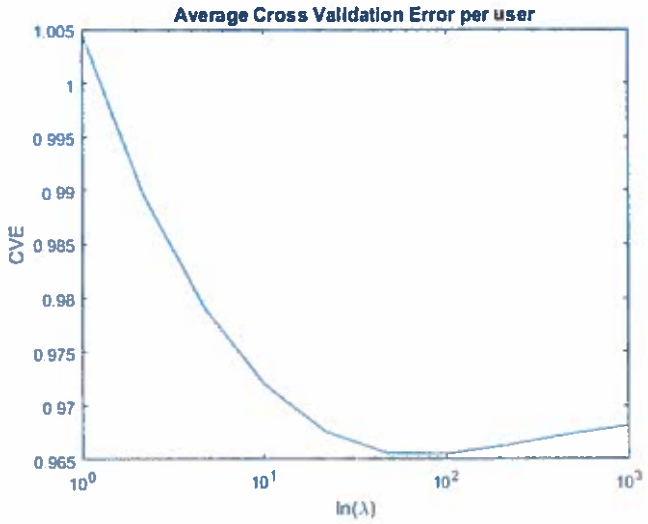



*Figure 8: Figure 9: Average CVE using Correlation and Legendre Non-Linear Transform*

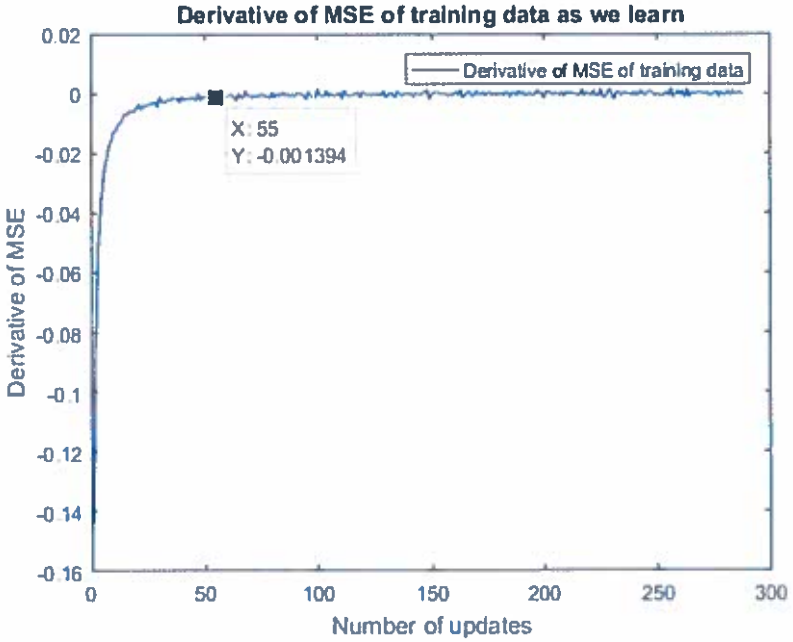



*Figure 10: Derivative of the training error as we learn using Collaborative filtering. We can see how it almost stops changing after around 55 iterations.*

# References

*StackExchange*. (2017, 02 28). Retrieved from http://math.stackexchange.com/questions/113270/the-median-minimizes-the-sum-of-absolute-deviations

final : 29 out of 32

Imperial College London

Imperial College London

# Assignment 3

Introduction to Machine Learning

p 2 : 3.5

p 3 : (b) 6

(c) 9

Nicolas Perez
3-6-2017
CID: 00931920

# *Non-plagiarism Pledge*

*I, Nicolas Perez, pledge that this assignment is completely my own work, and that I did not take, borrow or steal work from any other person, and that I did not allow any other person to use, have, borrow or steal portions of my work. I understand that if I violate this honesty pledge, I am subject to disciplinary action pursuant to the appropriate sections of Imperial College London.*

*Aspects of this coursework were discussed together with Aaron Low*

# Contents

Non-plagiarism Pledge ..... 1
Problem 1 ..... 3
Problem 2 ..... 4
Problem 3 ..... 6
(a) Constant base predictors ..... 6
(b) Linear Regression baseline ..... 6
(c) Linear regression with transformed features ..... 7
(d) Collaborative filtering ..... 8
Appendix ..... 10
References ..... 11

## Problem 1

**Problem 1** (Approximate ERM). Suppose you need to solve a binary classification problem with the binary error (i.e., the error is zero if the prediction is correct and 1 otherwise). Assume you are given data $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ drawn i.i.d. from the true data distribution. For any $\varepsilon \geq 0$, give a bound on the test error of any hypothesis $h_\varepsilon$ that is $\varepsilon$-optimal on the training set $\mathcal{D}$, that is, it satisfies $\widehat{R}_n(h_\varepsilon) \leq \varepsilon + \min_{h \in \mathcal{H}} \widehat{R}_n(h)$;[1] more precisely, give a high-probability upper bound on $R(h_\varepsilon) - \min_{h \in \mathcal{H}} R(h)$.

Let $h$ be a hypothesis in the hypothesis class $\mathcal{H}$. Then let's define $g$ and $h^*$ as:

$$g \in argmin(\hat{R}_n(h)) \qquad \textit{(def. 1)}$$

$$h^* \in argmin(R(h)) \qquad \textit{(def.2)}$$

That is, $g$ is the Empirical Risk Minimizer Hypothesis on the n-point training set $\mathcal{D} \subseteq \mathcal{X}$ and $h^*$ is the hypothesis with which minimises the test error on the set $\mathcal{X}$.

Then for any $\varepsilon \geq 0$ we have a set of hypothesis $\mathcal{H}_\varepsilon \subseteq \mathcal{H}$ which are $\varepsilon$-optimal on $\mathcal{D}$. This means that any $h_\varepsilon$ in this set satisfy

$$\hat{R}_n(h_\varepsilon) \leq \varepsilon + min_{h \in \mathcal{H}} \hat{R}_n(h) \qquad \textit{(eq. 1)}$$

According to the *def. 1* we have that *eq.1* is equivalent to $\hat{R}_n(h_\varepsilon) \leq \varepsilon + \hat{R}_n(g)$. This means that $h_\varepsilon$ is any hypothesis whose training error is at most $\varepsilon$ above the training error of the ERM hypothesis.

We want to bound the following expression

$$R(h_\varepsilon) - min_{h \in \mathcal{H}} R(h)$$

This can also we written as $R(h_\varepsilon) - min_{h \in \mathcal{H}} R(h) = R(h_\varepsilon) - R(h^*)$ using *def.2*

Thus, we want to bound how much worse would be the test error in case we choose any of the hypothesis $h_\varepsilon$ compared to the test error obtained when choosing $h^*$.

We have that

$$R(h_\varepsilon) - R(h^*) = R(h_\varepsilon) - \hat{R}_n(h_\varepsilon) + \hat{R}_n(h_\varepsilon) - \hat{R}_n(g) + \hat{R}_n(g) - \hat{R}_n(h^*) + \hat{R}_n(h^*) - R(h^*)$$

We can bound each two terms of the sum the following way

1. First term $R(h_\varepsilon) - \hat{R}_n(h_\varepsilon)$ we use the Vapnik-Chervonenkis Inequality which bound with a probability of at least $1 - \delta_1$

$$R(h_\varepsilon) - \hat{R}_n(h_\varepsilon) \leq \sqrt{\frac{8}{n} \ln\left(\frac{4 m_{\mathcal{H}}(2n)}{\delta_1}\right)} = \sqrt{\frac{8 d_{VC}}{n} \ln(2n+1) + \frac{8}{n} \ln\left(\frac{4}{\delta_1}\right)}$$

2. The second term $\hat{R}_n(h_\varepsilon) - \hat{R}_n(g)$ is, as stated above, equivalent to *eq.1*. Thus:

$$\hat{R}_n(h_\varepsilon) - \hat{R}_n(g) \leq \varepsilon$$

3. The third term is $\hat{R}_n(g) - \hat{R}_n(h^*) \leq 0$ according to the definition of $g$ (the training error of $g$ n the training set $\mathcal{D}$ is less than that of any of the other hypothesis).
4. Finally, the fourth term, $\hat{R}_n(h^*) - R(h^*)$ can be bounded using Hoeffding inequality which bound with a probability of at least $1 - \delta_2$

$$\hat{R}_n(h^*) - R(h^*) \leq \sqrt{\frac{1}{2n}\ln\left(\frac{2}{\delta_2}\right)}$$

Putting everything together gives that with a probability of $1 - \delta_1 - \delta_2$:

$$R(h_\varepsilon) - R(h^*) \leq \sqrt{\frac{8d_{VC}}{n}\ln(2n+1) + \frac{8}{n}\ln\left(\frac{4}{\delta_1}\right)} + \sqrt{\frac{1}{2n}\ln\left(\frac{2}{\delta_2}\right)} + \varepsilon$$

## Problem 2

**Problem 2** (Optimal predictor for the absolute error). Given data points $y_1, \ldots, y_n \in \mathbb{R}$, find a value $m$ minimizing the average absolute error $\frac{1}{n}\sum_{i=1}^{n}|y - y_i|$.

First, without loss of generalization, we can index the elements $y_i \in \mathbb{R}$ so that they are ordered. That is $y_1 \leq y_2 \leq \cdots \leq y_n$.

Then, we define the function $f(y) = \frac{1}{n}\sum_{i=1}^{n}|y - y_i|$.

If we evaluate the function $f$ on a $y \leq y_1$, (that is a $y$ that is smaller than all $y_i$) then as $y$ increases each term of $f(y)$ decreases at least until $y$ reaches $y_1$. That is $f(y_1) < f(y)$ for all $y < y_1$.

Then if we evaluate the function on $y + \varepsilon$, with $\varepsilon > 0$ so that $y_k \leq y \leq y + \varepsilon \leq y_{k+1}$ then

$$\begin{aligned} n \cdot f(y+\varepsilon) &= \sum_{i=1}^{k}(y + \varepsilon - y_i) + \sum_{i=k+1}^{n}(y_i - (y+\varepsilon)) \\ &= k\varepsilon + \sum_{i=1}^{k}(y - y_i) - (n-k)\varepsilon + \sum_{i=k+1}^{n}(y_i - y) \\ &= (2k-n)\varepsilon + \sum_{i=1}^{k}(y - y_i) + \sum_{i=k+1}^{n}(y_i - y) \\ &= (2k-n)\varepsilon + nf(y) \end{aligned}$$

We can the write

$$f(y+\varepsilon) - f(y) = \frac{(2k-n)\varepsilon}{n}$$

This difference is negative if $2k < n$, it is zero if $2k = n$ and positive if $2k > n$. Then in the interval $[y_k, y_{k+1}]$ the function $f(y)$, with $y_k \leq y \leq y_{k+1}$ behaves in the following way:

$$f(y)\begin{cases} decreases\ if\ 2k < n \\ constant\ if\ 2k = n \\ increases\ if\ 2k > n \end{cases}$$

This means that as $y$ moves from $-\infty$ to $+\infty$ the function will decrease and then increase and the minimum will be found in the interval $[y_k, y_{k+1}]$ where $2k = n$.

We need to consider now two possibilities.

> **Even $n$.** Then $k = \frac{n}{2}$. This means that $m$, the value that minimises $f$ is in the interval $[y_{n/2}, y_{n/2+1}]$. That is $m \in [y_{n/2}, y_{n/2+1}]$. All the values in this interval minimise $\frac{1}{n}\sum_{i=1}^{n}|y - y_i|$. We can take for example the average value between these two $m = \frac{y_{n/2}+y_{n/2+1}}{2}$
>
> **Odd $n$.** Then we can write $n = 2p + 1$, where $p \in \mathbb{N}$ and then $\frac{n}{2} = p + 1/2$. Then we must have that $y_k = y_{p+1/2}$ and $y_{k+1} = y_{p+3/2}$. These two values are between $y_p$ and $y_{p+2}$. Thus, we can take $m = y_{p+1}$ as our minimizer, that is, the value in the middle of the set of n-points.

In conclusion, we have found a set of values that minimise $\frac{1}{n}\sum_{i=1}^{n}|y - y_i|$ which is $[y_k, y_{k+1}]$ where $2k = n$. However, since we only need to pick one we can just pick the following

$$m = \begin{cases} \frac{y_{n/2} + y_{n/2+1}}{2}, & if\ n\ even \\ y_{p+1}, & if\ n\ is\ odd \end{cases}$$

> Where $n = 2p + 1$, where $p \in \mathbb{N}$ for the odd case.

This is the standard definition of the median for and even number of points (i.e. the average between the middle values) and for an odd number of points (i.e. value in the middle).

Why $y_n < y$ cannot increase $f(y)$?

3.5

# Problem 3

## (a) Constant base predictors

Firstly, we will provide a rating prediction for any movie (independent of the user) and for any user (independent of the movie). We will use the mean as a predictor. That is, each user rating for any movie will be predicted using the mean of ratings he has given so far and each movie rating for any user will be predicted using the mean of all the users that have rated it. We will then compare how well this works. In addition to this, since there are some movies that have not been rated in the training set but that may have been rated in the testing set we will give them a rate of 2.25 which is the average rating a movie can have. Choosing this value is arbitrary, nevertheless, the results do not change in any statistically meaningful way by choosing any value in the range rating 0.5-5.

In order to compare these two different ways of predicting the ratings we will obtain the Mean Squared Error (MSE) of all the ratings in the training and in the test sets: $MSE = \frac{1}{n}\sum_{i=1}^{n}(r_i - \hat{r}_i)^2$, where $n$ is the number of points on the training or test set. We will use this MSE from now on to quantify and compare how well the different learning methods we try behave.

| Learning Strategy | MSE on Training Set | MSE on Test Set |
|---|---|---|
| **Averaging user ratings** | 0.9108 | 0.9269 |
| **Averaging movie ratings** | 0.7851 | 1.0348 |

We see that the training error is closer to the test error when using the user rating average. It also gives a smaller test error compared to averaging the movie rating. Thus, we would choose using the user ratings rather than the movie ratings.

## (b) Linear Regression baseline

**Linear Regression.** The next learning strategy will consist on using Linear Regression to estimate the ratings based on some movies features $V$. Each user will have a vector $u_i$ which multiplied by the feature $v_j$ for a movie $j$ will give a prediction of the rating for that movie and that user $\hat{r}_{ij}$. The result of this is

| Learning Strategy | MSE on Training Set | MSE on Test Set |
|---|---|---|
| **Linear Regression** | 2.6116 | 3.2073 |

**Normalised Linear Regression.** This result is worse than that obtained using the constant predictors. It will probably improve if we normalize the movie features associated to a user so that each feature is centred and has a standard deviation of 1. In addition, the ratings for that user will be centred too. Then, when testing, we shift back the resulting rating by adding the corresponding mean for that user. The results are the following:

| Learning Strategy | MSE on Training Set | MSE on Test Set |
|---|---|---|
| **Normalised Lin REG** | 0.7068 | 1.0162 |

**Ridge Regression.** There is a noticeable improvement on the results but yet it is not as good as the constant base predictors. There is still a big gap between the error on training and on testing. This suggest that we may be overfitting the data. In order to fix this, we will use Ridge Regression. This way we will penalize the modulus of the weighting vector associated to each user, $u_i$. That is, we minimize $\hat{R}_n(u_i) + \frac{\lambda}{n}\|u_i\|^2$.

In order to choose the parameter $\lambda$ for ridge regression we use cross validation. In particular, leave-one-out cross validation. That is, we will partition the training set leaving one point out that we will use to obtain the validation error. We repeat this process so that we validate with all the points in the training set. Then we obtain the cross-validation error as the average of the validation errors (which are just the MSE of the validation set). By repeating this with several $\lambda$ values we can choose which one is the best for a particular user. Thus, we will get the optimal value of $\lambda_i$ for each user $i$.

The initial set of values of $\lambda$ to be tested will be logarithmically distributed between 0.0001 and 1000 (`logspace(-4,3,10)`) since, in principle, we do not expect $\lambda$ to be very large. This gives the following results.

| Learning Strategy | MSE on Training Set | MSE on Test Set |
|---|---|---|
| **Ridge Regression** | 0.8007 | 0.9072 |

This result is better than that obtained for the constant base predictors. It is also very noticeable how the difference between the training and test errors has reduced which means that we have reduced the overfitting. However, most of the $\lambda_i$ values we obtained through cross-validation are rather large (in the order of 100) as can be seen in the figure on the right.

That means that the modulus of the weighting vectors was indeed way larger than required and the data was being overfitted. We will then distribute the values initial set of $\lambda$ to be tested to 10 to 1000 (`linspace(10,1000,25)`). This will allow to better choose $\lambda$ (the number of $\lambda s$ to choose from in the optimal interval [10,1000] is larger) After doing this we get some improvement on the results:

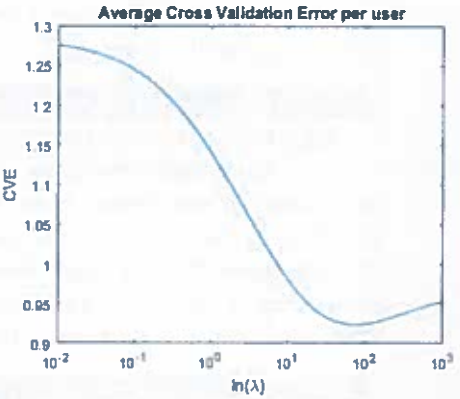


*Figure 1: Average CVE using Ridge Regression*

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Ridge Reg with optimal initial $\lambda$ set** | 0.7778 | 0.8834 |

**Lasso Penalty.** We can also try a different penalization. Instead of penalizing the square of the modulus of the weighting vectors we can penalize the absolute sum of its components. That is, a Lasso Penalization. We test several initial set of $\lambda$ as we did before and we find here that the region which gives the best results is from 0.01 to 1000 (`logspace(-2,3,100)`). However, this results in a worse outcome.

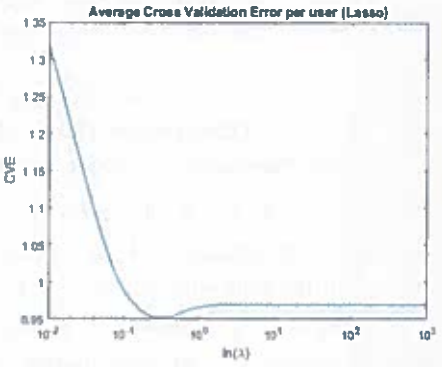


*Figure 2: Average CVE using Lasso Penalty*

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Lin Reg with LASSO penalty** | 0.7908 | 0.8941 |

## (c) Linear regression with transformed features

**PCA.** The objective now is to find a non-linear transformation that improves the results obtained so far. However, let's first try to do a Principal Component Analysis (which is a linear transformation) to try to reduce the features into a set of linearly uncorrelated variables and see if the result improves that way. We will do this with an initial set of $\lambda$ from 0.0001 to 1000 since we do not know which ones are going to work the best this time. The result obtained I the following:

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Ridge Regression after PCA features** | 0.7763 | 0.8923 |

Using PCA on the features reduces the number of features from 18 to 16. However, the outcome shows almost no improvement compared to the previous results.

**Interaction.** What we will try now is using the standard MATLAB function `x2fx()` which allows to perform an interaction non-linear transformation. That is, the output features will be all the possible products of each pair of input features (*i.e.* $x = (x_1, x_2, \dots, x_{18}) \rightarrow z = (x_1x_2, x_1x_3, \dots, x_{17}x_{18})$). There will be no pure quadratic features created since the features are binary and thus, squaring them will not have any effect. The results show no improvement and we can see in the fig. 7 in Appendix how the optimal values of $\lambda$ are even larger than those obtained before (as expected sin now the features vector has more components).

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Ridge Regression After interaction transformation** | 0.6814 | 0.9169 |

**Legendre.** The interaction transformation increases the number of features which is something that, in principle, we do not want to do since it may result in overfitting the training set. Thus, we will try to use a non-linear transformation which reduces the number of features. We will try the following, for each feature we will obtain its Legendre polynomials up to order 5. Then we will add all the new features of the same polynomial order. This means that we will end up with 6 features per movie. The results of both, test and training datasets

of this transformation are worst which suggest that we are underfitting the data. (Results of Ridge regression are on fig. 8 on the Appendix)

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Ridge Regression After Legendre transformation** | 0.8929 | 0.9115 |

**Correlation.** Finally, we will try to measure how relevant each feature is for each user. We can do this by taking the dot product of each feature with the ratings associated to that user. This will give a set of coefficients that will reflect how correlated each of them is to the rating. Then we will multiply each feature by this coefficient. This will boost those coefficients more correlated to the ratings. We should notice however that this is not a non-linear transformation, thus, after boosting the features we will perform the Legendre transformation described above. (Results of Ridge regression are on fig. 9 on the Appendix)

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Correlation and Legendre transformation** | 0.8957 | 0.9062 |

After trying all these non-linear transformations, we can conclude that none of them improve the results obtained using Ridge Regression on the features directly. In addition to this non-linear transformations more transformations using Legendre polynomials have been tested without any improvement.

## (d) Collaborative filtering

**Collaborative Filter - Gradient Descent.** The last approach that we will take will be learning simultaneously the movies features $v_j$ for all $n$ movies and the users weights $u_i$ for all $m$ users. We want to minimise the error $\hat{R}_N(u_i, v_j) = \frac{1}{2}\sum_{i,j}^{n,m}(r_{ij} - u_i{}^T v_j)^2 + \frac{\lambda}{2}\|v_j\|^2 + \frac{\lambda}{2}\|u_i\|^2$. If we store these vectors in the matrices $U_{mxK}$ and $V_{nxK}$ and the ratings in the matrix $r_{nxm}$ then, we can minimise $\hat{R}_N$ by updating the matrices in the following way $U_{t+1} = U_t - \eta\nabla U_t$ *where* $\nabla U_t = \sum_{i,j}^{n,m}(r - U_t V_t{}^T .* D)V_t + \lambda U_t$ and similarly, $V_{t+1} = V_t - \eta\nabla V_t$ *where* $\nabla V_t = \sum_{i,j}^{n,m}(r - V_t U_t{}^T .* D)U_t + \lambda V_t$. We have obtained the gradient from the partial derivative of the error function with respect to $U$ and $V$. The matrix $r$ will have the ratings centred per user (ignoring those movies not rated by the user). In order to not include those pairs $(i, j)$ with no rating associated we multiply element-wise with the matrix $D$ which has a 1 if a pair was rated and a 0 if a pair was not.

This method gives the following results when used with the learning rate $\eta = 0.001$, the maximum number of iterations set to 1000, $\lambda = 0$ and the dimension of the vectors, $K = 1$. These values were chosen after trying the algorithm a few times to get a reasonable $\hat{R}_{N,training}$ at the end (a value of $\eta > 0.002$ makes the error increase very fast instead of decrease).

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Collaborative Filter - Gradient Descent** | 0.7212 | 0.8453 |

We can see how the training and test error vary as we learn in the figure on the right. We notice that we did not choose the best possible maximum number of cycles since the minimum test error has not been reached yet. However, we only know this after looking at the test error which is something we cannot do beforehand. If we were to modify the algorithm because of the results we have seen on this test set we would be doing data snooping. This means that we would get a better result for this particular test set but we may do worse with others.

What we can do however is select the optimal parameters using cross validation or other similar method. However, this algorithm takes a very long time to run. Thus, we will try implement a faster version using a Stochastic Gradient Descent algorithm and use cross validation then.

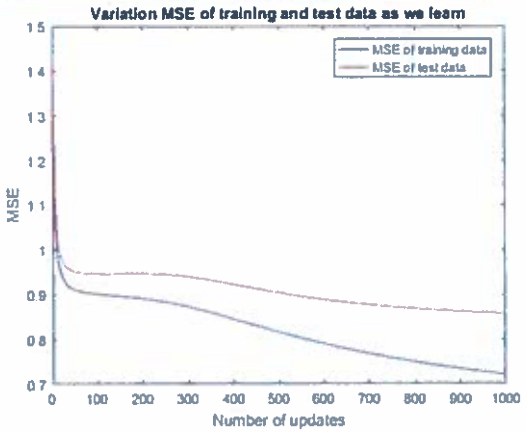



*Figure 3: Variation of MSE on the test and training data*

**Collaborative Filter – Stochastic Gradient Descent.** This algorithm uses the partial derivatives at a particular, randomly chosen, point to estimate the gradients. That is, $u_{i,t+1} = u_{i,t} - \eta \nabla u_{i,t}$ *where* $\nabla u_{i,t} = (r_{ij} - u_{i,t}{}^T v_{j,t}) v_j + \lambda u_{i,t}$ and similarly for $v_{j,t+1}$. Here we iterate 350 times through all the points (i.e. $N = 70002$ points, which is the number of valid pairs $(i, j)$) and we use $\eta = 0.001$. These two values were found by trying to reduce the training error on a trial and error basis. Then, we use cross validation dividing the $N$ points into 10 partitions from which we randomly take 6000 for training and 1000 for validation. The cross validation error is obtained as in question 3*b* and 3*c*. This gives the optimal number of features is 14 ($K = 14$) and the optimal $\lambda = 0.2$ using cross validation simultaneously on both parameters. This results can be seen on the figures on the right. We used $K = [2, 3, 5, 7, 9, 10, 12, 14, 16, 18, 20]$ and $\lambda = [0.0001, 0.001, 0.01, 0.1, 0.2, 0.4, 0.6, 0.8, 1]$

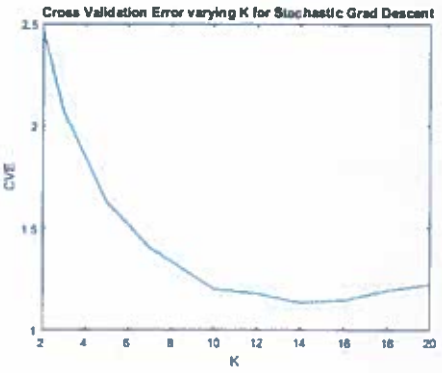



*Figure 4: Average CVE as we vary the number of features*

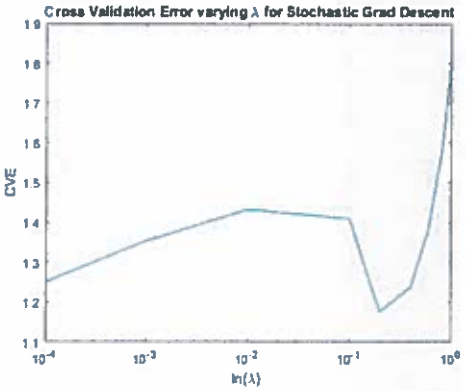


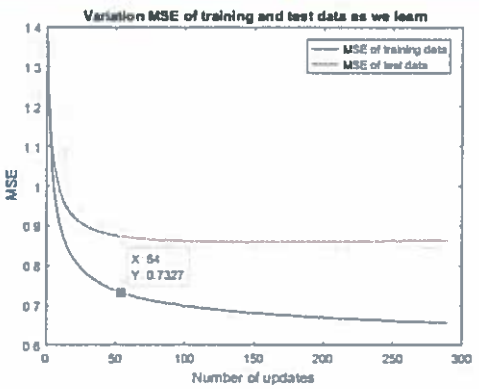

*Figure 5: Average CVE as we vary λ*

The best way of optimally choosing all the parameters would have been doing cross validation for all of them. However, we have not done this here due to the computational power required to evaluate all these parameters together. With the parameters considered above we get the following MSE.

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Stochastic Gradient Descent** | 0.6506 | 0.8542 |

We can plot the evolution of the training and test error as well as the evolution of the average modulus of the gradient defined as $\|\nabla \hat{R}_{n,}\| = \frac{1}{N}\left|\sum_{i,j}^{n,m}\left(\sum_{k=1}^{K} u_{i,k} + \sum_{k=1}^{K} v_{j,k}\right)\right|$, where $N$ is the total number of training points. This gives the results shown on fig. 6. From these we see two things. Firstly, we have not chosen the best maximum number of iterations since the minimum test error was reached before, this means we are slightly overfitting the data. Secondly, $\|\nabla \hat{R}_{n,}\|$ decreases as we update the vectors. We can use $\|\nabla \hat{R}_{n,}\|$ to stop the algorithm earlier. This would save us running time. We can see that the training error decreases slower when $\|\nabla \hat{R}_{N,}\| < 0.006$. We could use this threshold to stop the algorithm, however, in practice, we choose a threshold smaller (0.002) to avoid stopping the algorithm too early due to the oscillation of $\|\nabla \hat{R}_{N,}\|$. This gives the results shown in the table below.

| Learning Strategy | MSE ON Training Set | MSE ON Test Set |
|---|---|---|
| **Stochastic Gradient Descent with Stop** | 0.6884 | 0.8577 |



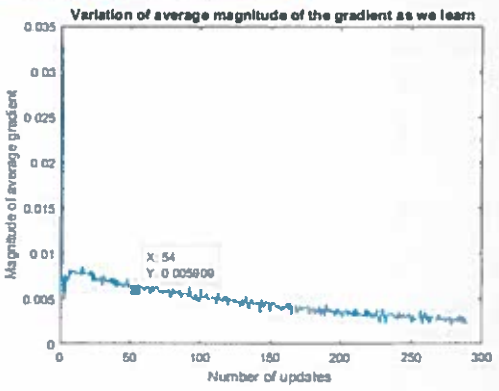




*Figure 6 :On the left, variation of MSE on the test and training data. On the right variation of the magnitude of the gradient.*

Alternatively we can approximate the derivative of the training error using the function `diff()` and stop the algorithm whenever is close enough to zero (this can be seen fig. 10 on the Appendix). We see however that even with cross validation, this algorithm does not improve the error obtained with the Non-Stochastic Gradient Descent.

# Appendix

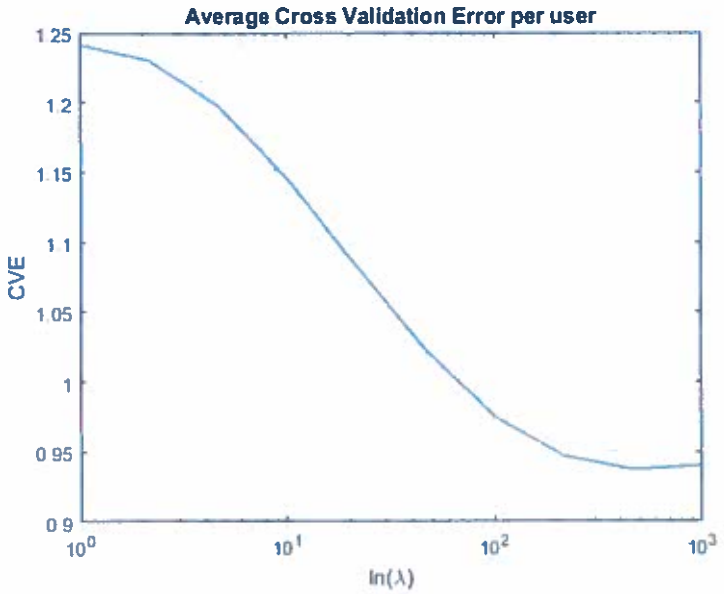



*Figure 7: Average CVE using Interaction Non-Linear Transform*

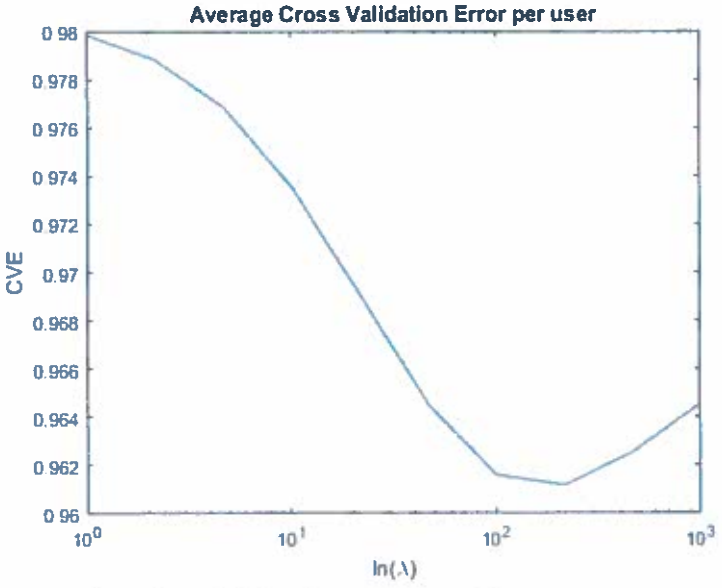



*Figure 8: Average CVE using Legendre Non-Linear Transform*

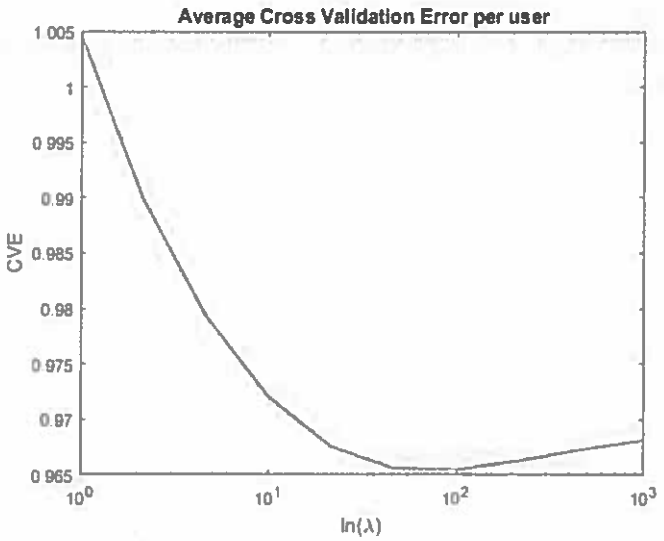



*Figure 8: Figure 9: Average CVE using Correlation and Legendre Non-Linear Transform*

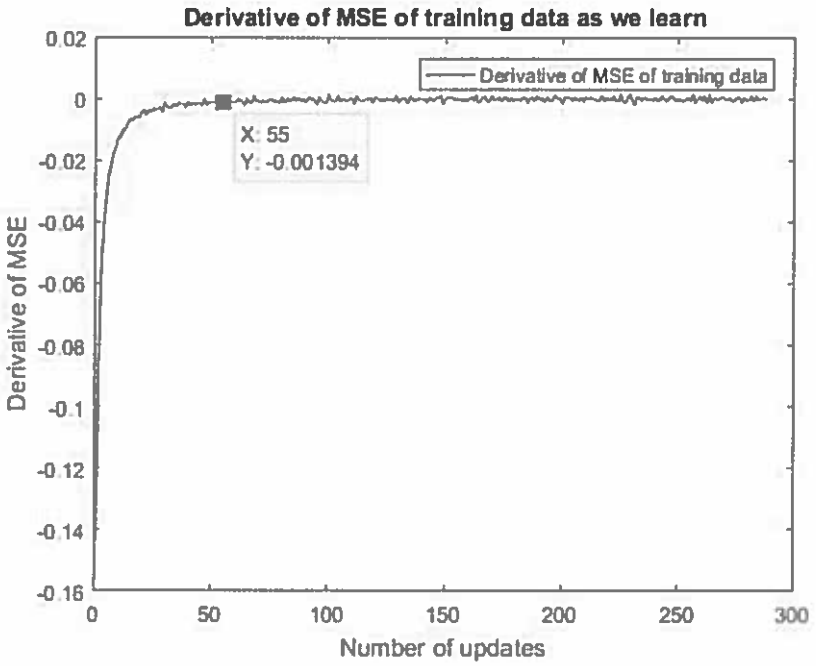



*Figure 10: Derivative of the training error as we learn using Collaborative filtering. We can see how it almost stops changing after around 55 iterations.*

# References

*StackExchange*. (2017, 02 28). Retrieved from http://math.stackexchange.com/questions/113270/the-median-minimizes-the-sum-of-absolute-deviations